Machine Learning II

Lecture 6: Attention and Self-Attention

Alexander Quispe
ENEI – INEI · PEU-CD 2026

October 23, 2026

Outline

Recap & Motivation

The bottleneck of seq2seq

Where We Left Off

Lectures so far:

- L1–L3: MLP, backprop, regularization.
- L4: CNNs for spatial data.
- L5: RNN, LSTM, GRU for sequential data.
- L6–L7: latent factor models, embeddings (lookup and learned).

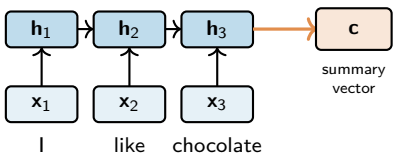
Key idea from L7: embed each token into a vector. A sentence becomes a sequence of vectors.

Today's question: when a model processes a sequence, how should each position decide *which other positions to look at*?

Attention is the answer. It started as a fix for RNN seq2seq, then took over deep learning.

Mini-Recap: What Is an Encoder?

Encoder = read the input sequence and build hidden states.

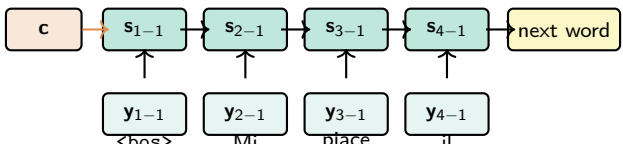


$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}; \mathbf{w}_e), \qquad \mathbf{c} = \mathbf{h}_T.$$

The encoder does not translate yet. It turns the source sentence into a sequence of memories, ending with a summary.

Mini-Recap: What Is a Decoder?

Decoder = generate the output sequence one token at a time.



$$\mathbf{s}_t = g(\mathbf{y}_{t-1}, \mathbf{s}_{t-1}, \mathbf{c}; \mathbf{w}_d), \qquad p(\mathbf{y}_t \mid \mathbf{y}_{<t}, \mathbf{x}_{1:T}) = \text{softmax}(W_o \mathbf{s}_t).$$

The decoder is a conditional language model: it writes the target sentence while conditioned on $\mathbf{c}$.

Numerical Mini-Example: Decoder Softmax

Suppose the decoder must choose the next Italian word after "Mi piace il".

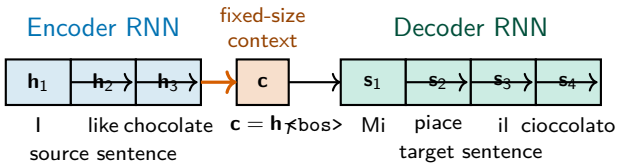
candidate word	cioccolato	cane	perché
logit z_i	2.0	1.0	0.0
$\exp(z_i)$	7.39	2.72	1.00
probability	0.67	0.24	0.09

$$\text{softmax}(2,1,0) = \frac{(e^2, e^1, e^0)}{e^2 + e^1 + e^0} \approx (0.67, 0.24, 0.09).$$

Interpretation: the decoder is not directly outputting a word; it outputs a probability distribution over the vocabulary.

RNN Seq2Seq Recap

Seq2seq idea: encode the source sentence into one vector, then decode the target sentence from it (Sutskever et al. 2014, Cho et al. 2014).



Equations:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}; \mathbf{w}_e) \qquad \text{encode}$$

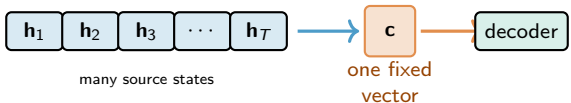
$$\mathbf{c} = \mathbf{h}_T \qquad \text{fixed context vector}$$

$$\mathbf{s}_t = g(\mathbf{y}_{t-1}, \mathbf{s}_{t-1}, \mathbf{c}; \mathbf{w}_d) \qquad \text{decode}$$

Limitation: all source information must pass through one vector $\mathbf{c}$. This works for short sentences, but becomes a bottleneck.

The Bottleneck Problem

Empirical observation (Bahdanau, Cho, Bengio 2014): translation quality *drops sharply* for long source sentences.



The information bottleneck: a fixed-dim vector $\mathbf{c} \in \mathbb{R}^d$ must summarize the whole source sentence.

Why this is hard for RNNs to fix on their own:

- Information from $\mathbf{x}_1$ has to flow through $\mathbf{h}_2, \mathbf{h}_3, \ldots, \mathbf{h}_T$ before reaching $\mathbf{c}$.
- Word-by-word alignment between source and target is not captured.

Idea (Bahdanau et al. 2014): let the decoder *look back* at the encoder hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_T$ – not just the final $\mathbf{c}$.

Today's Roadmap (Annotated)

1. **Bahdanau attention.** Derive attention as a fix for the seq2seq bottleneck.
2. **Query-Key-Value.** The abstraction that generalizes attention.
3. **Self-attention.** What if Q, K, V all come from the same sequence?
4. **Why $\sqrt{d_k}$?** The math behind scaled dot-product attention (CLT derivation – needed for HW3).
5. **Multi-head attention.** Many attention computations in parallel.
6. **Practical implementation.** PyTorch, masking, batch dims.
7. **Bridge to Transformers.** What's missing for Lecture 9.

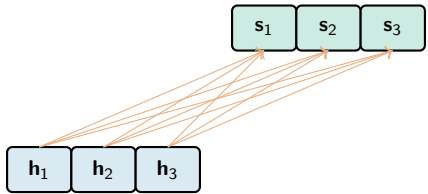
Bahdanau Attention

Three ideas, one mechanism

Idea 1: Symmetrize Input and Output

Old seq2seq: decoder uses only the final encoder state $\mathbf{c}$.

New idea: decoder uses *all* encoder states $\mathbf{h}_1, \ldots, \mathbf{h}_T$.



every decoder step sees every encoder state

Simplest version: the new context is just the average of encoder states:

$$\mathbf{c} = \frac{1}{T} \sum_{\tau=1}^{T} \mathbf{h}_\tau.$$

But this loses positional information. Each output position uses the same average. Bad.

Idea 2: Make Context Specific to Each Output

Improvement: let each decoder step t compute its *own* weighted average over encoder states.

Per-output context:

$$\mathbf{c}_t \;=\; \sum_{\tau=1}^{T} \alpha_{t,\tau}\, \mathbf{h}_\tau .$$

where $\alpha_{t,\tau} \in \mathbb{R}$ tells us how much output position t should attend to input position τ.

Question: how do we set $\alpha_{t,\tau}$?

Answer: learn it from the data, as a similarity between the decoder state $\mathbf{s}_{t-1}$ and the encoder state $\mathbf{h}_\tau$.

Simplest similarity: dot product.

$$e_{t,\tau} \;=\; \langle \mathbf{s}_{t-1}, \mathbf{h}_\tau \rangle .$$

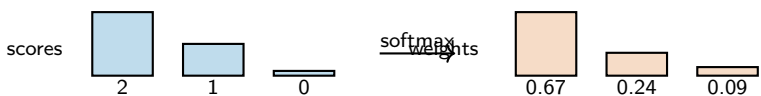
Still missing: the $e_{t,\tau}$ are raw scores, not probabilities. They can be negative, unbounded, hard to interpret.

Idea 3: Normalize via Softmax

Convert scores to probabilities using softmax over the input positions:

$$\alpha_{t,\tau} = \frac{\exp(e_{t,\tau})}{\sum_{\tau'=1}^{T} \exp(e_{t,\tau'})}.$$

Now $\alpha_{t,\tau} \geq 0$ and $\sum_{\tau} \alpha_{t,\tau} = 1$.



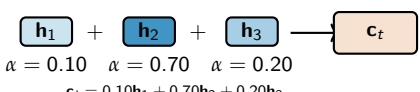
Softmax turns raw scores into an alignment distribution.

Bahdanau Attention Recipe

At each decoder step t, repeat the same three operations:

score: $e_{t,\tau} = \langle \mathbf{s}_{t-1}, \mathbf{h}_\tau \rangle$ normalize: $\alpha_{t,\tau} = \mathrm{softmax}_\tau(e_{t,\tau})$

average: $\mathbf{c}_t = \sum_\tau \alpha_{t,\tau} \mathbf{h}_\tau$



Then decode using this context: $\mathbf{s}_t = g(\mathbf{y}_{t-1}, \mathbf{s}_{t-1}, \mathbf{c}_t; \mathbf{w}_d)$.

Next: rename this pattern as query, key, value.

Numerical Mini-Example: Weighted Context

Suppose the decoder puts most attention on the second source word.

$$\mathbf{h}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{h}_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \qquad \mathbf{h}_3 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \alpha = (0.10, 0.70, 0.20).$$

$$\begin{aligned} \mathbf{c}_t &= 0.10\mathbf{h}_1 + 0.70\mathbf{h}_2 + 0.20\mathbf{h}_3 \\ &= 0.10 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0.70 \begin{bmatrix} 0 \\ 2 \end{bmatrix} + 0.20 \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0.10 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1.40 \end{bmatrix} + \begin{bmatrix} 0.40 \\ 0.20 \end{bmatrix} = \begin{bmatrix} 0.50 \\ 1.60 \end{bmatrix}. \end{aligned}$$

Interpretation: attention builds a new vector by mixing source states, not by choosing only one.

Worked Example: "Boys' Age?"

Setup (from Caltech CS148): we have five people.

Name (**key**)	Joseph	Wendy	Nellie	Mary	Theodore
Boy? (1=yes)	1	0	0	0	1
Age (**value**)	11	5	8	3	7

Query: "what is the average age of the *boys*?"

The query says "look at boys". The keys say who is a boy. The values are the ages.

Compute weights (hard-attention intuition: keep boys, mask others):

$$\alpha = (\,0.5,\ 0,\ 0,\ 0,\ 0.5\,) \quad \text{after masking + normalization.}$$

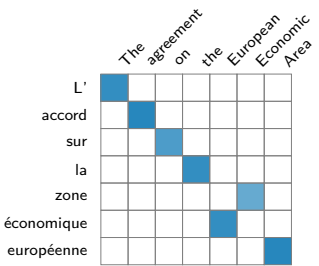
Weighted sum of values:

$$0.5 \cdot 11 \ + \ 0 \cdot 5 \ + \ 0 \cdot 8 \ + \ 0 \cdot 3 \ + \ 0.5 \cdot 7 \ = \ 9.$$

Query asks, keys match, values are averaged.

Visual: Bahdanau Attention Heatmap

Real attention (Bahdanau et al. 2014, En-Fr):



Reading the heatmap:

Bright cells = high $\alpha_{t,\tau}$.

The diagonal pattern shows mostly-monotonic alignment, with *local reordering*:

"European Economic Area"
$\leftrightarrow$ "zone économique européenne".

No alignment supervision – it emerges from end-to-end training.

Section Summary

What changed relative to old seq2seq?

- Old decoder: every output uses the same fixed vector $\mathbf{c} = \mathbf{h}_T$.
- Attention decoder: each output gets its own context $\mathbf{c}_t$.
- The weights $\alpha_{t,\tau}$ act like a soft alignment between target position t and source position τ.

decoder state $\mathbf{s}_{t-1}$ asks a question; encoder states $\mathbf{h}_\tau$ answer it.

Coming up: rename these roles as query, key, and value.

Query, Key, Value

The general abstraction

Three Roles in Attention

Reread Bahdanau attention with new names:

Role	Bahdanau	What it does
Query q	$\mathbf{s}_{t-1}$ (decoder state)	"what am I looking for?"
Key $\mathbf{k}_\tau$	$\mathbf{h}_\tau$ (encoder state)	"what do I represent for matching?"
Value $\mathbf{v}_\tau$	$\mathbf{h}_\tau$ (encoder state)	"what do I contribute if matched?"

Same recipe, now in Q/K/V language:

$$e_\tau = \langle \mathbf{q}, \mathbf{k}_\tau \rangle \qquad \text{score query against each key}$$

$$\alpha_\tau = \mathrm{softmax}_\tau(e_\tau) \qquad \text{normalize across positions}$$

$$\mathbf{o} = \sum_\tau \alpha_\tau \, \mathbf{v}_\tau \qquad \text{average the values}$$

In Bahdanau, key = value = encoder hidden state. Later we'll let them differ.

Numerical Mini-Example: Q/K/V Dot Products

Let the query ask for "boy-like" records.

$$\mathbf{q} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \mathbf{k}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{k}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{k}_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

$$e_1 = \mathbf{q}^\top \mathbf{k}_1 = 1, \qquad e_2 = \mathbf{q}^\top \mathbf{k}_2 = 2, \qquad e_3 = \mathbf{q}^\top \mathbf{k}_3 = 3.$$

$$\mathrm{softmax}(1, 2, 3) \approx (0.09, 0.24, 0.67).$$

If the values are ages $\mathbf{v} = (11, 5, 7)$, then

$$\mathbf{o} = 0.09(11) + 0.24(5) + 0.67(7) \approx 6.88.$$

Interpretation: bigger dot product $\Rightarrow$ bigger attention weight $\Rightarrow$ bigger influence on the output.

The Boys' Age Example, Restated

Think of attention as a soft database lookup.

	Joseph	Wendy	Nellie	Mary	Theodore
key: boy?	1	0	0	0	1
value: age	11	5	8	3	7
weight α	0.5	0	0	0	0.5

Query q: "average age of boys?" **Output:** $0.5(11) + 0.5(7) = 9$.

Mechanism:

1. Score: $\mathbf{q} \cdot \mathbf{k}_\tau$ is large when person τ matches the query "boy".
2. Softmax: convert scores to a probability distribution over the five people.
3. Output: $\sum_\tau \alpha_\tau \mathbf{v}_\tau$ = expected age *under that distribution*.

The same machinery handles any "soft lookup" problem: the query is the question, keys advertise content, values are what gets returned.

Matrix Form: Shapes

Stack queries, keys, values into matrices.

For n queries, m key/value pairs, key-dim d_k, value-dim d_v:

	Shape	Description
Q	$n \times d_k$	one row per query
K	$m \times d_k$	one row per key
V	$m \times d_v$	one row per value
O	$n \times d_v$	one row per output

$\mathbf{QK}^\top$ compares every query row to every key row.

Numerical Mini-Example: Matrix Scores

Two queries, three keys, dimension $d_k = 2$.

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 2}, \qquad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2}.$$

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 3}.$$

Interpretation: each row is one query's scores against all three keys.

Matrix Form: Many Queries at Once

The vector recipe becomes three matrix operations.

Score matrix:

$$\underbrace{\mathbf{Q}}_{n\times d_k}\underbrace{\mathbf{K}^\top}_{d_k\times m}=\underbrace{\mathbf{E}}_{n\times m}.$$

Attention weights: softmax *row-wise* (each query is normalized independently):

$$\mathbf{A}=\mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\in\mathbb{R}^{n\times m}.$$

Output:

$$\underbrace{\mathbf{A}}_{n\times m}\underbrace{\mathbf{V}}_{m\times d_v}=\underbrace{\mathbf{O}}_{n\times d_v}.$$

Scaled Dot-Product Attention

The canonical formula (Vaswani et al. 2017):

Scaled dot-product attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Why the name "scaled dot-product"?

- "Dot product" = the score $\mathbf{Q}\mathbf{K}^\top$.
- "Scaled" = the $1/\sqrt{d_k}$ factor.

Why scale by $\sqrt{d_k}$? We'll derive this in Section 5 via the Central Limit Theorem.

This single formula is the workhorse of every modern Transformer.

Cross-Attention vs Self-Attention

So far: queries come from one source, keys/values from another.

- Q = decoder states, K = V = encoder states.
- Called **cross-attention** (or encoder-decoder attention).

Next: what if Q, K, V all come from the *same* sequence?

- Q = K = V = encoder states.
- Called **self-attention**.
- Every position attends to every other position in the same sequence.

Why this is the key idea: self-attention lets us *replace* the recurrent encoder. No more sequential bottleneck.

Self-attention is the building block of every modern language model.

Self-Attention

When Q, K, V come from the same sequence

Self-Attention: Definition

Input: a sequence $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T]^\top \in \mathbb{R}^{T \times D}$ where $\mathbf{x}_i$ is the embedding of position i.

Three learned projections – the heart of self-attention:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K^\top, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_V^\top.$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_k \times D}$ and $\mathbf{W}_V \in \mathbb{R}^{d_v \times D}$.

Apply scaled dot-product attention:

$$\mathbf{O} \;=\; \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \;\in\; \mathbb{R}^{T \times d_v}.$$

Reading the formula:

- Same input $\mathbf{X}$ on all three sides.
- Three different views via the projections.
- Each output row is a context-aware representation of one input position.

Why Three Different Projections?

Each role demands different content from $\mathbf{x}_i$:

- As a **query**: "what am I looking for in other positions?"
- As a **key**: "what do I advertise for matching with other queries?"
- As a **value**: "what content do I contribute when matched?"

Without separate projections, we would force the same vector to play all three roles. The model couldn't, e.g., advertise a different aspect (key) than what it contributes (value).

Concrete example. The word *bank*:

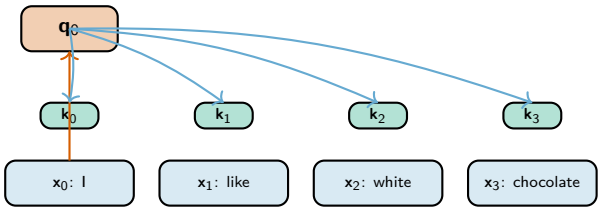
- As a query, it might ask "is there a financial context?"
- As a key, it advertises "I'm a noun that could be financial or geographic."
- As a value, it contributes a representation that gets combined with context.

The three projections specialize at training time.

Build-up: One Query at a Time

Take sentence "I like white chocolate" ($T = 4$). Build attention step by step.

Query $\mathbf{q}_0$ from "I" attends to all keys $\mathbf{k}_0, \mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3$



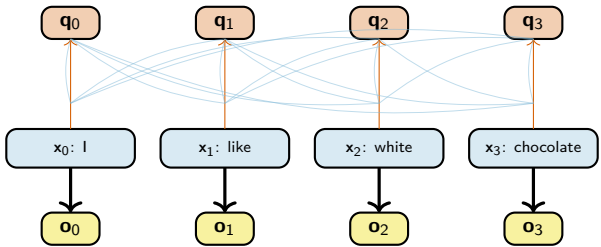
For position 0 ("I"):

- Project: $\mathbf{q}_0 = \mathbf{W}_Q \mathbf{x}_0$, $\mathbf{k}_j = \mathbf{W}_K \mathbf{x}_j$, $\mathbf{v}_j = \mathbf{W}_V \mathbf{x}_j$ for all j.
- Score: $e_{0,j} = \mathbf{q}_0^\top \mathbf{k}_j / \sqrt{d_k}$.
- Weights: $\alpha_{0,j} = \mathrm{softmax}_j(e_{0,j})$.
- Output: $\mathbf{o}_0 = \sum_j \alpha_{0,j} \mathbf{v}_j$.

Build-up: Every Position Attends to Every Other

Repeat the same procedure for $\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3$.

$T^2 = 16$ pairwise score computations



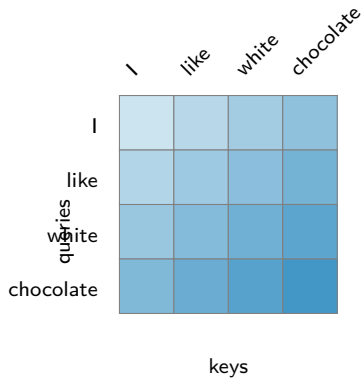
In matrix form, all of this is just one operation:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

The whole sentence is processed **in parallel**, not sequentially like an RNN.

Self-Attention as a Matrix

The attention matrix has one row per query and one column per key.



Row *i* is a probability distribution: which tokens should position *i* read from?

Self-Attention vs CNN vs RNN

Receptive field comparison. How does each layer connect position i to position j?

Property	RNN	CNN	Self-attention
Path length, $i \to j$	$O(\lvert i-j \rvert)$	$O(\log_k T)$	$O(1)$
Parallelism	sequential	parallel	parallel
Receptive field per layer	1 step	kernel size	full sequence
Inductive bias	order	locality	permutation-equivariant

What the table says:

- RNN needs $\lvert i-j \rvert$ steps to connect distant positions.
- CNN needs log depth (with dilated convs).
- Self-attention connects all positions in one step, but needs positional encoding for order.

Self-attention trades inductive bias for flexibility.

Computational Complexity

Per-layer cost for a sequence of length T and embedding dim D:

Layer type	Compute	Comment
RNN	$O(T \cdot D^2)$	dominated by recurrent matmul
CNN (kernel k)	$O(k \cdot T \cdot D^2)$	still linear in T
Self-attention	$O(T^2 \cdot D)$	quadratic in T

Crossover. Self-attention is cheaper than RNN when $T < D$ (typical for sentences with $T \approx 50$, $D \approx 512$).

For very long sequences ($T \gg D$, e.g. entire documents): self-attention becomes expensive. This motivates:

- Sparse attention (Longformer, BigBird).
- Linear attention (Performer, Linformer).
- Flash attention (efficient implementation).

For typical NLP tasks, T^2 lets every word talk to every other.

Section Summary

Self-attention in one slide:

1. Take an input sequence $\mathbf{X} \in \mathbb{R}^{T \times D}$.
2. Project three ways: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q^\top$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K^\top$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V^\top$.
3. Compute pairwise scores: $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T \times T}$.
4. Normalize: $\text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})$.
5. Aggregate: multiply by $\mathbf{V}$.

Properties:

- Every position connects to every other in one step ($O(1)$ path length).
- Fully parallel.
- No notion of order (yet – Lecture 9 fixes this with positional encoding).

Coming up: we still owe an explanation of the $\sqrt{d_k}$ scaling. It's not cosmetic.

Why $\sqrt{d_k}$?

The Central Limit Theorem in action

The Softmax Saturation Problem

Reminder. Softmax with input scale matters a lot:

Logits	Softmax	Comment
$(1, 0, 0)$	$(0.58, 0.21, 0.21)$	gentle, gradients flow
$(5, 0, 0)$	$(0.99, 0.007, 0.007)$	sharper
$(20, 0, 0)$	$(\approx 1, 10^{-9}, 10^{-9})$	saturated, gradients vanish

Why gradients vanish at saturation. The softmax Jacobian has entries

$$\frac{\partial \,\mathrm{softmax}(\mathbf{z})_i}{\partial z_j} = \mathrm{softmax}(\mathbf{z})_i \cdot (\delta_{ij} - \mathrm{softmax}(\mathbf{z})_j).$$

When one entry is ≈ 1 and the rest are ≈ 0, every derivative is ≈ 0.

We need to make sure the inputs to softmax stay at a reasonable scale – typically $O(1)$.

Setup: What Are the Dot-Product Scores?

Assumption (reasonable at initialization): entries of $\mathbf{q}$ and $\mathbf{k}$ are i.i.d. with mean 0 and variance 1.

$$q_i \sim \mathcal{N}(0,1), \qquad k_i \sim \mathcal{N}(0,1), \qquad q_i \perp k_j.$$

Dot product:

$$s = \mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i.$$

Goal. Compute the distribution of s.

Why this matters. The score s feeds directly into softmax. If s has wide spread, softmax saturates.

CLT Derivation: One Term

Step 1: Mean of each term.

$$\mathbb{E}[q_i k_i] \;=\; \mathbb{E}[q_i] \cdot \mathbb{E}[k_i] \;=\; 0 \cdot 0 \;=\; 0.$$

(Using independence.)

Step 2: Variance of each term.

$$\begin{aligned} \mathrm{Var}(q_i k_i) &= \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 \\ &= \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] - 0 \\ &= 1 \cdot 1 \;=\; 1. \end{aligned}$$

CLT Derivation: Sum of Terms

Step 3: Variance of the sum. Since the d_k terms are independent,

$$\mathrm{Var}(s) \;=\; \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) \;=\; d_k.$$

Step 4: Apply CLT. For large d_k,

$$s \;\sim\; \mathcal{N}(0, d_k) \quad \text{approximately.}$$

Standard deviation: $\sqrt{d_k}$.

Why This Is Bad Without Scaling

Typical values of d_k in modern Transformers:

- BERT-base: $d_k = 64$, so $\sqrt{d_k} = 8$.
- GPT-3 small heads: $d_k = 128$, so $\sqrt{d_k} \approx 11$.
- Large models: $d_k = 256$, so $\sqrt{d_k} = 16$.

Consequence. Scores typically range over $\pm 2\sqrt{d_k}$, so ± 16 for BERT-base.

Plugging into softmax: the largest logit dominates by a factor of $e^{16} \approx 9 \times 10^6$. Output is near one-hot. Gradients vanish.

Empirical signature without scaling: training stalls within the first hundreds of steps; loss plateaus at random.

The fix is one line of code: divide by $\sqrt{d_k}$ before softmax.

The Fix: Divide by $\sqrt{d_k}$

Define the scaled score:

$$\tilde{s} = \frac{s}{\sqrt{d_k}} = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}.$$

Variance of the scaled score:

$$\mathrm{Var}(\tilde{s}) = \frac{\mathrm{Var}(s)}{d_k} = \frac{d_k}{d_k} = 1.$$

So: $\tilde{s} \sim \mathcal{N}(0, 1)$ approximately. Softmax inputs stay at $O(1)$ scale, gradients flow.

Scaled dot-product attention

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

The $\sqrt{d_k}$ normalizer keeps softmax inputs at unit variance.

Numerical Mini-Example: Why Scaling Helps

Use a common head dimension: $d_k = 64$, so $\sqrt{d_k} = 8$.

	without scaling	**with scaling**
raw dot product	16	16
score into softmax	16	$16/8 = 2$
example logits	$(16, 0, 0)$	$(2, 0, 0)$
softmax output	$(\approx 1, 10^{-7}, 10^{-7})$	$(0.79, 0.11, 0.11)$

$$\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}} = \frac{16}{8} = 2.$$

Interpretation: scaling prevents one key from winning too aggressively at initialization, so gradients remain useful.

Multi-Head Attention

Many attention computations in parallel

Why Multiple Heads?

One head learns one kind of attention pattern.

Examples of patterns that may emerge:

- Head A: attend to syntactic dependencies (subject $\leftrightarrow$ verb).
- Head B: attend to coreference (pronoun $\leftrightarrow$ antecedent).
- Head C: attend to nearby words (local structure).
- Head D: attend to entities mentioned earlier in the document.

One single attention map cannot capture all of these.

Solution. Run several attention computations in parallel – one per "head" – and combine the outputs.

Multi-head attention = ensemble of attention computations with different learned projections.

Multi-Head: The Math

Setup. Number of heads h. Each head has its own projection matrices.

For head $i = 1, \ldots, h$:

$$\mathbf{Q}^i = \mathbf{X}\mathbf{W}_Q^{i,\top}, \qquad \mathbf{K}^i = \mathbf{X}\mathbf{W}_K^{i,\top}, \qquad \mathbf{V}^i = \mathbf{X}\mathbf{W}_V^{i,\top}.$$

Per-head attention:

$$\mathbf{O}^i \;=\; \mathrm{softmax}\!\left(\frac{\mathbf{Q}^i(\mathbf{K}^i)^\top}{\sqrt{d_k}}\right)\mathbf{V}^i \;\in\; \mathbb{R}^{T\times d_v}.$$

Concatenate the head outputs:

$$\mathbf{O}_{\text{cat}} \;=\; [\mathbf{O}^1, \mathbf{O}^2, \ldots, \mathbf{O}^h] \;\in\; \mathbb{R}^{T\times hd_v}.$$

Final output projection by $\mathbf{W}_O \in \mathbb{R}^{D\times hd_v}$:

$$\mathbf{O} \;=\; \mathbf{O}_{\text{cat}}\,\mathbf{W}_O^\top \;\in\; \mathbb{R}^{T\times D}.$$

HW3 note: track the concat-then-$\mathbf{W}_O$ step carefully.

Multi-Head: Shape Cheat-Sheet

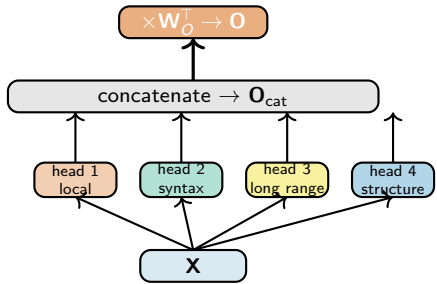
All shapes in one place:

Tensor	Shape	Description
$\mathbf{X}$	$T \times D$	input sequence
$\mathbf{W}_Q^i$, $\mathbf{W}_K^i$	$d_k \times D$	per-head Q/K projections
$\mathbf{W}_V^i$	$d_v \times D$	per-head V projection
$\mathbf{Q}^i$, $\mathbf{K}^i$	$T \times d_k$	per-head queries/keys
$\mathbf{V}^i$	$T \times d_v$	per-head values
$\mathbf{O}^i$	$T \times d_v$	per-head output
$\mathbf{O}_{\text{cat}}$	$T \times hd_v$	concatenated heads
$\mathbf{W}_O$	$D \times hd_v$	output projection
$\mathbf{O}$	$T \times D$	final output

Common choice: $d_k = d_v = D/h$, so the per-head dim is small and the total parameter count matches a single-head attention with dim D.

E.g., BERT-base: $D = 768$, $h = 12$, so $d_k = d_v = 64$.

Multi-Head: Visual



Reading the diagram:

- Same input $\mathbf{X}$ feeds all h heads (here $h = 4$).
- Each head can learn a different relation because it has its own $\mathbf{W}_Q^i, \mathbf{W}_K^i, \mathbf{W}_V^i$.
- Outputs are concatenated and projected by a single $\mathbf{W}_O$.

Section Summary

Multi-head attention recipe:

1. Project $\mathbf{X}$ into h different (Q, K, V) triplets, one per head.
2. Run scaled dot-product attention independently in each head.
3. Concatenate the h outputs.
4. Project back to dim D via $\mathbf{W}_O$.

What multi-head buys you:

- Each head can specialize on a different attention pattern.
- Same total compute as single-head with full dim (when $d_k = D/h$).
- Empirically much better than single-head.

Coming up: how do you actually implement this in PyTorch? And what about masks?

Practical Implementation

Shapes, masks, PyTorch

PyTorch: nn.MultiheadAttention

Built-in module: `torch.nn.MultiheadAttention`.

```python
import torch.nn as nn

mha = nn.MultiheadAttention(
    embed_dim=768,     # = D
    num_heads=12,      # = h, so d_k = d_v = 64
    batch_first=True   # use (batch, T, D) shape
)

# x: (batch=32, T=50, D=768)
out, attn_weights = mha(x, x, x)    # self-attention: Q=K=V=x
```

Shapes returned:

- `out`: `(32, 50, 768)` – same shape as input.
- `attn_weights`: `(32, 50, 50)` – attention map per batch element.

For cross-attention: `mha(query, key_value, key_value)`.

The module hides the projection matrices and the concat-then-W_O step.

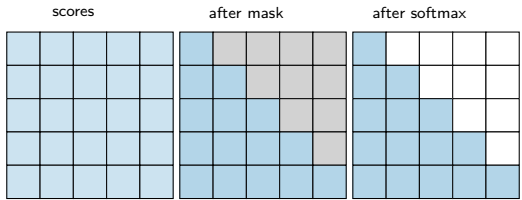
Causal Mask (for Language Models)

Problem. A decoder predicting the next word must not see future tokens.

Fix. Add a **causal mask** to the score matrix *before* softmax:

$$e_{i,j} = \begin{cases} \mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k}, & j \le i \\ -\infty, & j > i. \end{cases}$$

After softmax, the $-\infty$ entries become 0.



Position i can only attend to positions $\le i$.

Numerical Mini-Example: Causal Mask

Suppose position $i = 3$ scores five keys. Key 4 is in the future.

$$\text{raw scores} = (1.2,\ 0.5,\ 2.0,\ 0.7,\ 3.1).$$

$$\text{after causal mask} = (1.2,\ 0.5,\ 2.0,\ 0.7,\ -\infty).$$

$$\exp(1.2, 0.5, 2.0, 0.7, -\infty) \approx (3.32,\ 1.65,\ 7.39,\ 2.01,\ 0),$$
$$\text{softmax} \approx (0.23,\ 0.11,\ 0.51,\ 0.14,\ 0).$$

Interpretation: even if the future token had the largest raw score (3.1), the mask forces its probability to exactly zero.

Padding Mask (for Batched Sequences)

Problem. Batches contain sequences of different lengths, padded to the longest. We shouldn't attend to padding tokens.

Example batch:

- Sequence 1: "the cat sat" (length 3)
- Sequence 2: "hello" (length 1)
- Padded to length 5: `[the, cat, sat, PAD, PAD]` and `[hello, PAD, PAD, PAD, PAD]`.

Fix. Mask out the padding positions in keys (so no query attends to them):

$$e_{i,j} = -\infty \quad \text{if } j \text{ is a padding position.}$$

In PyTorch:

```
key_padding_mask = (input_ids == PAD_TOKEN)   # (batch, T) of bools
out, _ = mha(x, x, x, key_padding_mask=key_padding_mask)
```

Both masks (causal + padding) are commonly combined in language models.

Econ Applications & Bridge to Lecture 9

Attention for Time-Series Forecasting

Temporal Fusion Transformer (Lim et al., *IJF* 2021).

Architecture for forecasting macro time series:

- Embed each observation (numerical + categorical features).
- LSTM for short-range temporal patterns.
- Self-attention over the full history for long-range dependencies.
- Quantile loss for uncertainty estimates.

Why attention helps for macro forecasting:

- Past crises (e.g., 2008, 2020) are sparse, distant events.
- RNNs struggle to remember them; attention can directly reference them.
- Diagnostics: attention weights can highlight *which past periods the model uses*.

For econ: attention weights can be useful diagnostics, though not complete explanations.

Attention for Tabular Data

TabNet (Arik & Pfister, AAAI 2021) and **FT-Transformer** (Gorishniy et al., NeurIPS 2021).

Setup. Each row of a tabular dataset has features $(x_1, x_2, \ldots, x_F)$ – e.g., a credit application.

The trick. Embed each *feature* as a vector, then run self-attention over the feature embeddings.

The model learns which features matter for the prediction *conditionally on other features*.

Applied econ use cases:

- Default prediction with attention weights as interpretability.
- Demand forecasting with feature attention.
- Heterogeneous treatment effect estimation: which covariates moderate the treatment?

Caveat: for small-to-medium tabular data, gradient boosting (XGBoost, LightGBM) often still wins. Attention shines when you have $> 10^4$ rows and rich categorical features.

Attention for Text-as-Data in Econ

Modern econ NLP pipelines use Transformer encoders (BERT, RoBERTa, BETO for Spanish).

Typical workflow:

1. Take a corpus (FOMC minutes, central bank speeches, news, earnings calls).
2. Tokenize and feed through a pretrained Transformer.
3. Extract the contextual embeddings (one vector per token, or one per document).
4. Use those as features for downstream econ analysis.

Recent papers:

- Hansen, Kazinnik (2023): Fed sentiment from FOMC text via BERT embeddings.
- Bybee, Kelly, Manela, Xiu (2024, JFE): news topic attention for asset prices.
- Ash et al. (2024): judicial reasoning from opinions.

These all rely on attention's ability to produce context-dependent embeddings – the topic that's been our through-line.

What's Missing: Bridge to Lecture 9

We've built scaled dot-product self-attention. What's left to assemble a Transformer?

1. **Positional encoding.** Self-attention is permutation-equivariant – it has no notion of word order. We need to inject position information.
2. **Layer normalization.** Normalize each token's vector to stabilize training.
3. **Residual connections.** $\mathbf{O} = \text{Attention}(\mathbf{X}) + \mathbf{X}$. Helps gradient flow.
4. **Feed-forward sublayer.** A position-wise MLP after attention – adds capacity.
5. **Stack many blocks.** GPT, BERT, T5 are all just stacks of these blocks.

Lecture 9 puts the pieces together: *Attention Is All You Need* (Vaswani et al. 2017) and the modern Transformer architecture.

Summary & Next Steps

One Mechanism, Many Settings

The same attention recipe appeared all lecture:

- **Bahdanau attention:** decoder state queries encoder states.
- **Self-attention:** each token queries every token in the same sequence.
- **Multi-head attention:** repeat this in parallel with different projections.

Today's Big Ideas

1. **Bahdanau attention** solves the seq2seq bottleneck: let the decoder look at all encoder states with learned weights.
2. **Query-Key-Value** is the general abstraction. Q asks, K advertises, V contributes.
3. **Self-attention** reuses the same input for Q, K, V. It connects any two positions in one step.
4. **Scaled dot product.** The $\sqrt{d_k}$ factor keeps softmax inputs at unit variance (CLT argument).
5. **Multi-head attention** runs h attention computations in parallel and projects back through $\mathbf{W}_O$.
6. **Complexity tradeoff.** Self-attention is $O(T^2 D)$ vs RNN's $O(TD^2)$. Crossover at $T \approx D$.
7. **For econ:** attention gives interpretable, context-aware representations of text, time series, and tabular data.

Practical Recipes

When to use attention:

- Sequence length $T < D$: faster than RNN.
- Need to model long-range dependencies: better than RNN.
- Need interpretability via attention maps: clear win.

When to be careful:

- Very long sequences ($T > 10{,}000$): consider sparse / linear attention.
- Small training data: attention may underperform LSTMs or boosted trees.
- Time series with strong autoregressive structure: hybrid models often win.

Common pitfalls:

- Forgetting the $\sqrt{d_k}$ scaling – training will stall.
- Forgetting padding mask – the model attends to noise.
- Forgetting causal mask in a language model – it cheats and looks at future tokens.

Next Lecture: Transformers

Lecture 9:

- Positional encoding (sinusoidal and learned).
- Layer normalization and residual connections.
- The Transformer encoder and decoder blocks.
- *Attention Is All You Need* (Vaswani et al. 2017).
- BERT vs GPT: encoder-only vs decoder-only.
- Tokenization: byte-pair encoding.
- Pretraining and fine-tuning.

The big picture:

$$\textit{Embeddings} \rightarrow \textit{Attention} \rightarrow \textit{Self-attention} \rightarrow \textit{Transformer} \rightarrow \textit{LLMs}$$

Questions?

References

- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *ICLR*.
- Sutskever, I., Vinyals, O., & Le, Q. (2014). Sequence to sequence learning with neural networks. *NeurIPS*.
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
- Lim, B., Arik, S., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *IJF*.
- Arik, S., & Pfister, T. (2021). TabNet: attentive interpretable tabular learning. *AAAI*.
- Gorishniy, Y., et al. (2021). Revisiting deep learning models for tabular data. *NeurIPS*.
- Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). BERT: pre-training of deep bidirectional transformers. *NAACL*.
- Hansen, S., & Kazinnik, S. (2023). Can ChatGPT decipher Fedspeak? *Working paper*.
- Bybee, L., Kelly, B., Manela, A., & Xiu, D. (2024). Business news and business cycles. *Journal of Finance*.